Introduction to Programming:

CONSTANTS, VARIABLES, AND DATA TYPES

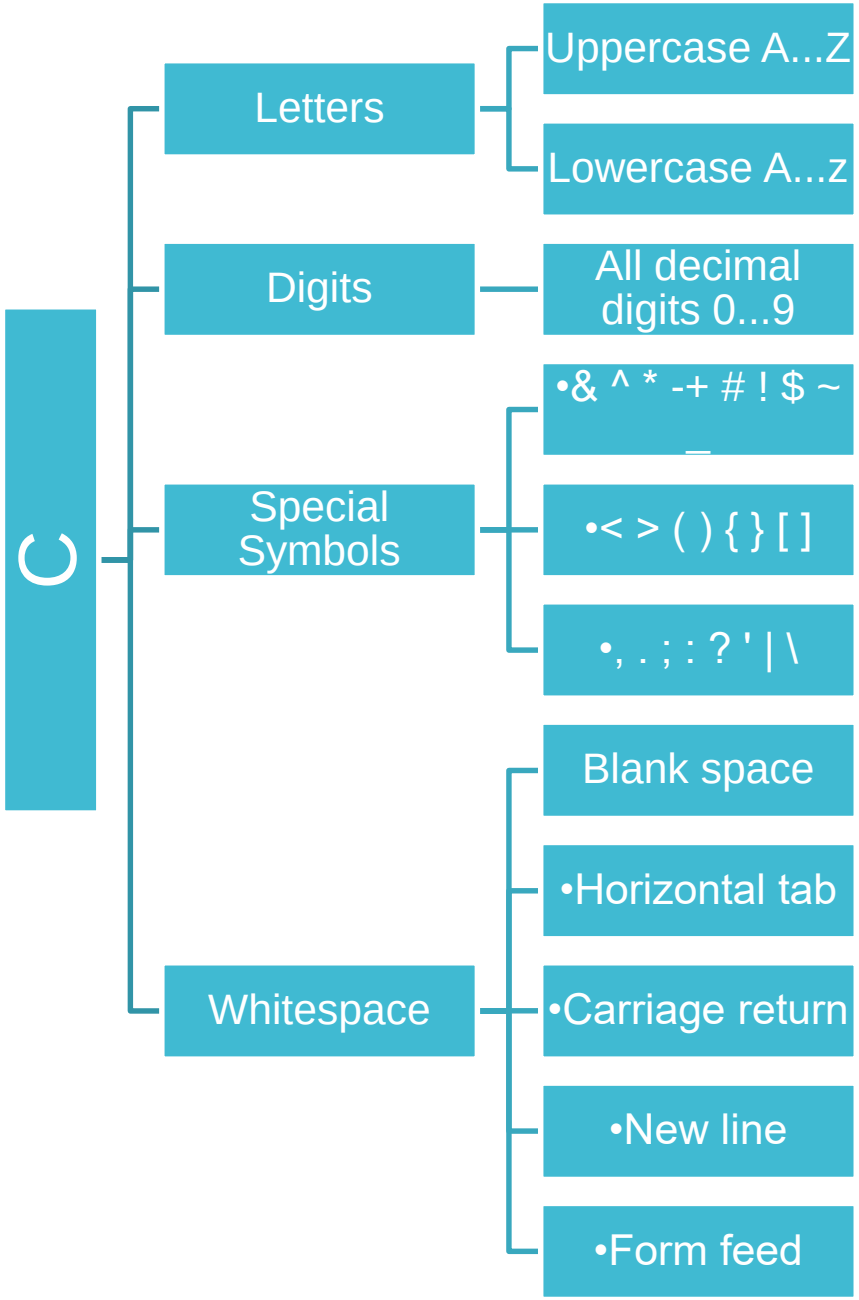
Prepared By:

Darsha Chauhan

Outline

- Character Set in C
- Tokens
  - Keywords
  - Identifiers
  - Constants
  - Strings
  - Operators
  - Special Symbols
- Data Types:
  - Primary Data Types and
  - User Defined Data Types
- Variables:
  - Declaration of Variables
  - Assigning Values to Variables
  - Initialization of Variables
  - Declaring Variables as Constants.
  - Defining Symbolic Constants

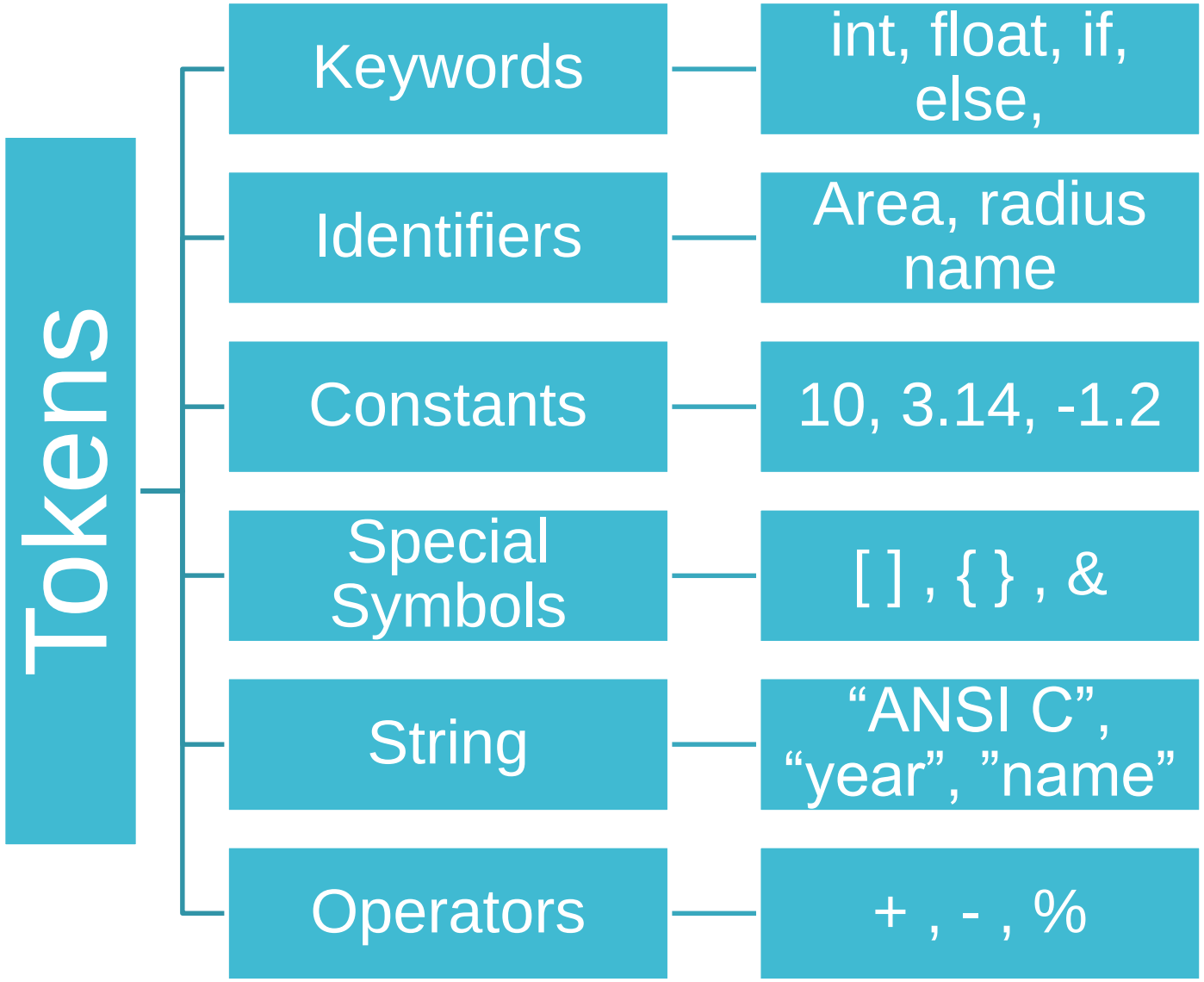
C Character Set



Tokens

- The smallest individual unit of a program is known as token.
- C programs are written using these tokens and syntax of the language.
- The classification of C tokens are as follows:

Tokens


Tokens
Keywords
int, float, if, else,
Identifiers
Area, radius name
Constants
10, 3.14, -1.2
Special Symbols
[ ] , { } , &
String
“ANSI C”, “year”, ”name”
Operators
+ , - , %

Tokens: Keywords

- C reserves a set of 32 words for its own use.
- These words are called keywords (or reserved words), and each of these keywords has a special meaning within the C language.
- Keywords are the words whose meaning has already been fixed to C compiler (or in broad sense to the computer).
- Keywords cannot be used as variable names because if we do so we are trying to assign a new meaning to the keyword, which is not allowed by the computer.
- Keyword serve as basic building blocks for computer statements.
- There are 32 keywords available in ANSI C as follows:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Tokens: identifiers

Identifiers:

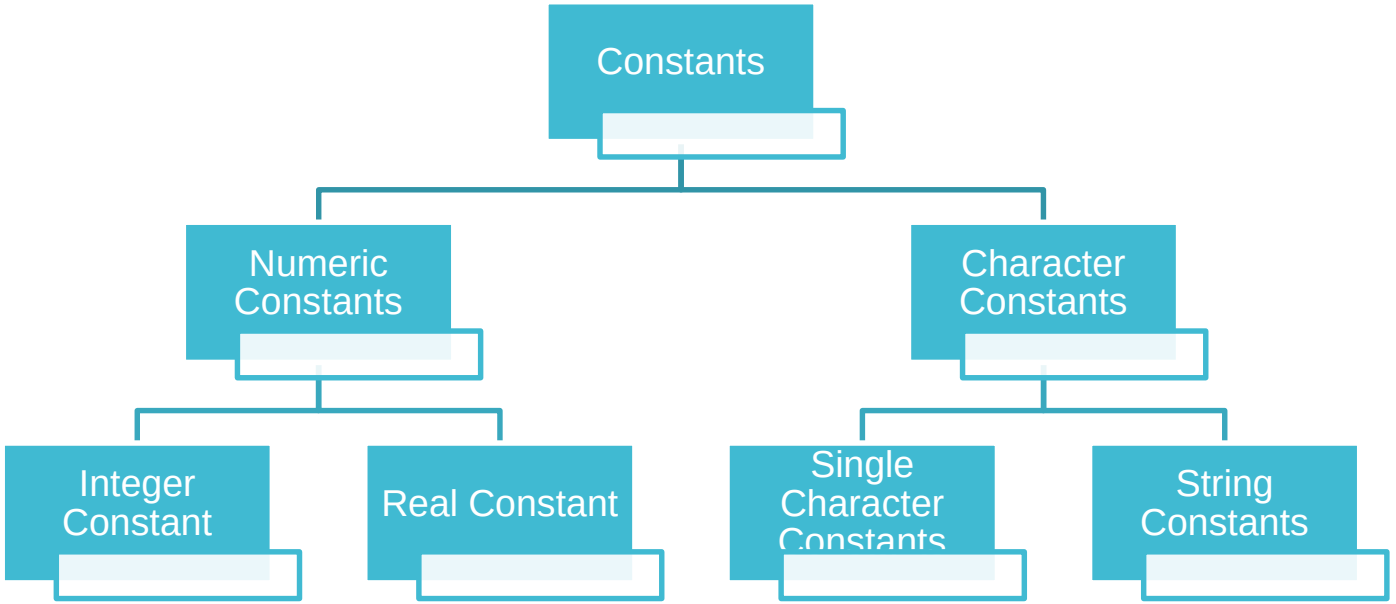
- Identifiers are names that are given to various user defined program elements, such as variable, function and arrays.
- These are user defined names consisting of sequence of letters and digits with either letter or the underscore (_) as first character.
- Both uppercase and lowercase are permitted, although lowercase letters are commonly used.
- The underscore character is also permitted in identifiers. It is usually used as a link between two words in long identifiers.
- Rules for identifier:
  - First character must be an alphabet or underscore_
  - Must consist of only letters, digits or underscore
  - Only first 31 characters are significant
  - Cannot be a keyword
  - Must not contain white space



Tokens: Constants

Constants:

- Constants refer to fixed values that do not change during execution of program.



Tokens: Constants: Integer

- Integer constant refers to a sequence of digits
- There are three types of integer constant.
- **Decimal Integer**:
  - It consists a set of digits(0 to 9), having optional + or – sign.
  - No other characters are allowed.
  - Example: 123, +25, -78.
- **Octal Integer**:
  - It consists of any combination of digit 0 to 7 and + or – sign.
  - Octal numbers are preceded by 0(zero).
  - For example: 037,-0551.
- **Hexadecimal Integer**:
  - It consists of any combination of digits from the set 0 to 9 and A to F alphabets. 'A' represents 10, 'B' represents 11, … , 'F' represents 12.
  - It always starts with 0x or 0X.
  - For example 0X2A, 0x95, 0xA47C.



Tokens: Constants: Character

Single Character Constant:

- It consists of single character enclosed within a pair of single quote mark.
- For example: ‘Z’, ‘a’, ‘+’, ‘ ’, ‘1’.

String Constant:

- A string constant is sequence of character enclosed within a double inverted coma.
- The characters may be letter, number, special character, blank space, etc.
- For example: “ANSI”, “1998”, “?A.B,”, “5+3” etc.
- ‘A’ is a character and “A” is a string.

Tokens

Strings:

- A string is a sequence of characters terminated with a null character \o.

Special Symbols:

- Symbols such as #, &, =, * are used in C for some specific function are called as special symbols.

Operators:

- An operator is a symbol that tells the compiler to perform certain mathematical or logical operation.



Trigraph Characters

- Many non-English keyboards do not support all characters mentioned above.
- ANSI C introduces the concept of "trigraph" sequences to provide a way to enter certain characters that are not available on some keyboards.
- Each trigraph sequence consists of three characters.

Trigraph Sequence	Translation
??=	# number sign
??(	[left bracket
??)	] right bracket
??<	{ left brace
??>	} right brace
??!	| vertical bar
??/	\ back slash
??-	~ tilde



Backslash Character Constant

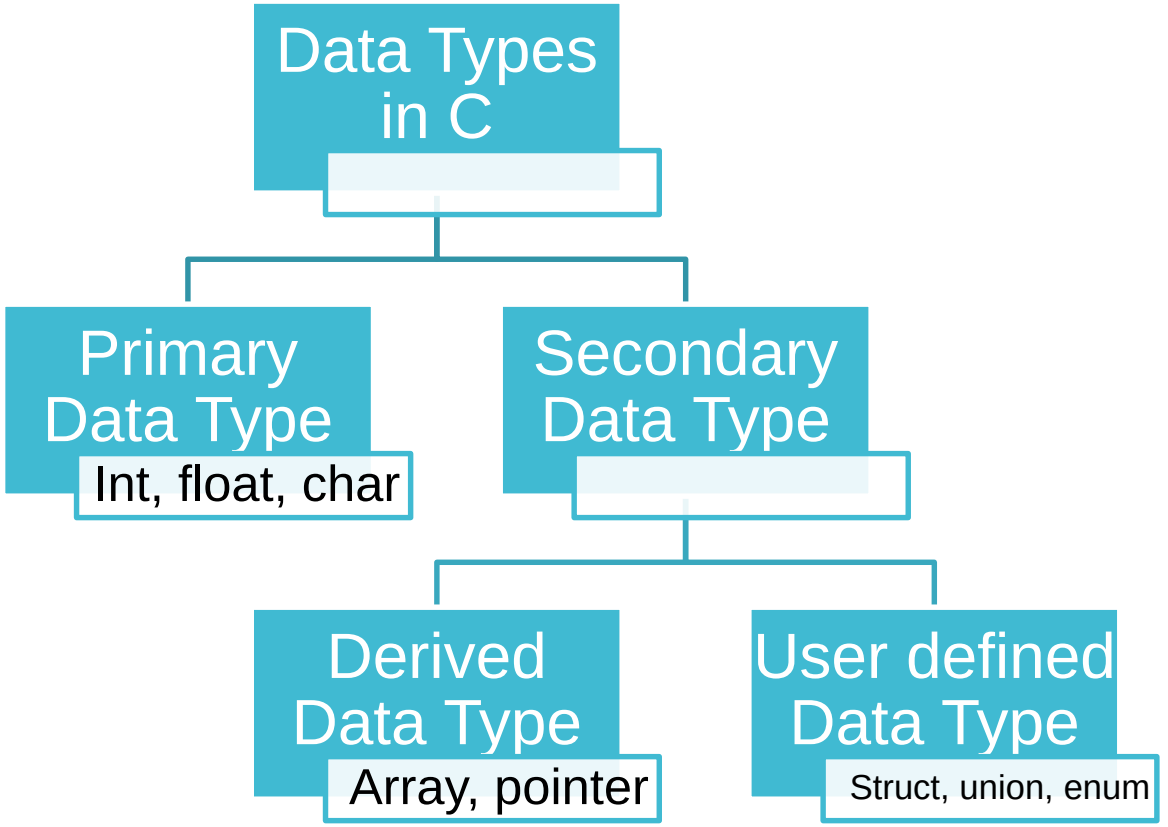
- C supports some backslash character constants that are used in output functions.
- Backslash character constants are a special type of character constant consists of two character.
- This is also known as escape sequence. Escape sequence starts with '\' (backslash character).

Backslash Character Constant

Backslash Character Constant	Meaning
'\a'	Audible alert
'\b'	Back space
'\f'	Form feed- takes control to next page
'\r'	Carriage return- takes control to next paragraph
'\n'	New Line
'\t '	Horizontal tab
'\v'	Vertical tab
'\' '	Single Quote
'\" '	Double quote
'\?'	Question mark
'\\'	Backslash
'\o'	End of string - Null

Data Types

- Data types are defined as the data storage format that a variable can store a data.
- It determines the type and size of data associated with variables.



Data Types

Data Type	Size in Bits (Bytes)	Range (Signed)	Range (Unigned)
short int	8 (1 byte)	-128 to 127	0 to 255
int	16 (2 bytes)	-32768 to 32767	0 to 65535
long int	32 (4 bytes)	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295
float	32 (4 bytes)	3.4E-38 to 3.4E+38	Precision: 6
double	64 (8 bytes)	1.7E-308 to 1.7E+308	Precision: 14
long double	80(10 bytes)	3.4E-4932 to 3.4E+4932	
Char	8 (1 byte)	-128 to 127	0 to 255



Data Types: Primary Data type

- Primary data types are built in data types which are directly supported by machine.
- They are also known as fundamental data types.

int:

- Integer numbers are whole number without decimal point or fractional parts. Example: 20, -154, 0, 77 etc.
- Signed number can store positive or negative values.(Example:20,-154)
- Unsigned number means value can be only zero or positive. Example: 0, 25, 100.

float:

- float data type can store floating point number which represents a real number with decimal point and fractional part such as 10.50, 155.25 etc.
- When the accuracy of the floating point number is insufficient, we can use the double to define the number. The double is same as float but with longer precision.
- To extend the precision further we can use long double which consumes 80 bits of memory space.
- Example: float a=10.50;



Data Types: Primary Data type

char:

- Char data type can store single character of alphabet or digit or special symbol such as ‘a’, ‘5’ etc.
- Each character is assigned some integer value which is known as ASCII values.
- Example: char a=‘a’;

void:

- The void type has no value therefore we cannot declare it as variable as we did in case of int or float or char.
- The void data type is used to indicate that function is not returning anything.



Data Types: Secondary Data type

- Secondary data types are not directly supported by the machine.
- It is combination of primary data types to handle real life data in more convenient way.
- It can be further divided in two categories,
- Derived data types:
- Derived data type is extension of primary data type. It is built-in system and its structure cannot be changed.
- Examples: Array and Pointer.
- Array: An array is a fixed-size sequenced collection of elements of the same data type.
- Pointer: Pointer is a special variable which contains memory address of another variable.



Data Types: Secondary Data type

User defined data types:

- User defined data type can be created by programmer using combination of primary data type and/or derived data type.
- Examples: Structure, Union, Enum.
- Structure: Structure is a collection of logically related data items of different data types grouped together under a single name.
- Union: Union is like a structure, except that each element shares the common memory.
- Enum: Enum is used to assign names to integral constants, the names make a program easy to read and maintain.



Data Types: Secondary Data type

Userdefined Type Declaration:

- C supports a feature known as "type definition" that allows users to define an identifier that would represent an existing datatype
- Syntax: typedef type identifier;
- Example: typedef float marks;

  marks sub1[50], suv2[50];o
- Enumerated datatype:
- Syntax: enum identifier{value1, value2, ..};
- Example: enum day{Monday,Tuesday, … sunday};



Variable

- A variable is a data name that may be used to store a data value.
- A variable may take different values at different times during execution.
- A variable name can be chosen by a programmer in a meaningful way so as to reflect its function or nature in the program.

Rules for declaring a variable:

- They must begin with a letter. Some system permit underscore as the first character.
- ANSI standard recognizes a length of 31 character. However, length should not be normally more than eight characters, since only first eight characters are treated as significant by many compiler. (In C99, at least 63 characters are significant.)
- Uppercase and lowercase are significant that is, the variable Total is not the same as total or TOTAL.
- It should not be a keyword.
- White space is not allowed.



Variable: Declaration

- After designing suitable variable names, we must declare them to the compiler.
- Declaration does two things:
  - It tells compiler what variable name is.
  - It specifies what type of data the variable will hold.
- The declaration of variables must be done before they are used in the program.

Syntax:

- data-type v1,v2,v3... vn;
- Here, v1, v2, v3, ... vn are the names of variables. Variables are separated by commas. A declaration statement must end with semicolon.
- For Example, valid declarations are:

```
int count;

int number, total;

float area;
```

Variable: Initialization

- **Variable Initialization: (Assigning values to variable)**
- Values can be assigned to variables using assignment operator = as follows:
- Variable_name = constant;

Declaration	Remarks
radius = 3;	
fact=1 ; count= 1;	C permits multiple assignments in one line.
int fact=1;	assigning a value to variable at the time variable is declared
fact=count=1	Initialization of more than one variable using multiple assignment operators.
data-type variable_name = constant; int fact=1;	The process of giving initial values to variable is called initialization. (Assigning a value to a variable at the time the variable is declared.)

Variable: Assigning values

- An assignment statement implies that the values of the variable on the left of the '=' sign is set equal to the value of the quantity (or the expression) on the right.
- Syntax:

  variable-name = constant;

- Example:

  Count=1;

  count=count+1;

- Above statement means the 'new value' of count is equal to 'old value' of count plus 1.
- During assignment operation, C converts the type of value on right-hand side to the type on the left. This may involve truncation when value is converted to an integer.
- External and static variables are initialized to zero by default.
- Automatic variables that are not initialized explicitly will contain garbage value.



Symbolic Constant

Constant:

- Constant is a fixed value which cannot be changed
- Syntax: const datatype const_name = value;
- For Example: const int a =5;

Symbolic Constant:

- Constant values are assigned to these name at the beginning of the program.
- Syntax:

  #define symbolic-name value of constant
- Valid example of constant definition are:

  #define STRENGTH 100

  #define PASS_MARK 50

  #define PI 3.14



Control String

- As we know in C language to access different types of data types, we have to use different keywords and different memory allocation is there as per data types.
- So like that, to access different kind of data types, we have to use different control strings, which are as follows:

Data Type	Control String
signed char/ unsigned char	%c
short signed int / signed int	%d
short unsigned int / unsigned int	%u
long signed int	%ld
long unsigned int	%lu
float	%f
double	%lf
long double	%LF
string	%s
octal int	%o
hexadecimal int	%x

#Thank You

From @:Darsha

☺☺☺